

PostgreSQL Extensions Overview

Table of Contents

1. [Learning Objectives](#)
 2. [Extension Architecture](#)
 3. [pg_stat_statements](#)
 4. [PostGIS](#)
 5. [pg_partman](#)
 6. [TimescaleDB](#)
 7. [pgvector](#)
 8. [pg_trgm](#)
 9. [uuid-ossdp / gen_random_uuid](#)
 10. [hstore](#)
 11. [pg_cron](#)
 12. [btree_gin / btree_gist](#)
 13. [Managing Extensions](#)
 14. [Production Monitoring Queries](#)
 15. [Common Mistakes](#)
 16. [Best Practices](#)
 17. [Interview Questions & Answers](#)
 18. [Exercises and Solutions](#)
 19. [Cross-References](#)
-

Learning Objectives

By the end of this module you will be able to:

- Explain the PostgreSQL extension architecture and lifecycle
 - Configure and use `pg_stat_statements` for query performance tracking
 - Choose between `pg_trgm` and FTS for similarity-based search
 - Use `pgvector` for ML embedding similarity search
 - Implement UUID generation strategies for distributed primary keys
 - Evaluate `hstore` vs `jsonb` for key-value storage
 - Manage extension versions and upgrades safely in production
-

Extension Architecture

PostgreSQL Extension System:

Extension Package (on disk):

```
/usr/share/postgresql/15/extension/
├─ pg_stat_statements.control  (metadata)
├─ pg_stat_statements--1.10.sql (install script)
├─ pg_stat_statements--1.9--1.10.sql (upgrade script)
└─ /usr/lib/postgresql/15/lib/
    └─ pg_stat_statements.so  (shared library)
```

CREATE EXTENSION

In Database (pg_extension catalog):

Registers: functions, operators, types, indexes
Creates: tables, views in extension schema
Tracks: version, dependencies

Query: SELECT * FROM pg_extension;

```
-- Basic extension management
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
DROP EXTENSION pg_stat_statements;
ALTER EXTENSION pg_stat_statements UPDATE;

-- Check installed extensions
SELECT extname, extversion, extrelocatable
FROM pg_extension
ORDER BY extname;

-- Check available (not yet installed) extensions
SELECT * FROM pg_available_extensions ORDER BY name;

-- Check available versions
SELECT * FROM pg_available_extension_versions WHERE name = 'pg_stat_statements';
```

pg_stat_statements

The single most important extension for PostgreSQL performance work. Tracks execution statistics for every unique query pattern.

Setup

```
-- 1. Add to postgresql.conf (requires server restart):
-- shared_preload_libraries = 'pg_stat_statements'
-- pg_stat_statements.track = all (top/all/none)
-- pg_stat_statements.max = 10000 (max queries tracked)

-- 2. Install in target database:
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Essential monitoring queries

```
-- Top 10 slowest queries by average execution time
SELECT
    ROUND(mean_exec_time::numeric, 2) AS avg_ms,
```

```

ROUND(total_exec_time::numeric, 2)      AS total_ms,
calls,
ROUND(stddev_exec_time::numeric, 2)      AS stddev_ms,
rows,
ROUND(100.0 * total_exec_time /
      SUM(total_exec_time) OVER (), 2)  AS pct_total,
LEFT(query, 80)                          AS query
FROM pg_stat_statements
WHERE query NOT LIKE '%pg_stat_statements%'
ORDER BY mean_exec_time DESC
LIMIT 10;

-- Top queries by total time (most impactful to optimize)
SELECT
calls,
ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
ROUND(mean_exec_time::numeric, 2)        AS avg_ms,
ROUND(rows::numeric / NULLIF(calls, 0), 0) AS avg_rows,
LEFT(query, 100)                         AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

-- Queries with high I/O (shared_blks_hit + read)
SELECT
query,
calls,
ROUND(mean_exec_time::numeric, 2) AS avg_ms,
shared_blks_hit,
shared_blks_read,
ROUND(shared_blks_hit::numeric /
      NULLIF(shared_blks_hit + shared_blks_read, 0) * 100, 1) AS cache_hit_pct
FROM pg_stat_statements
WHERE shared_blks_hit + shared_blks_read > 0
ORDER BY shared_blks_read DESC
LIMIT 10;

-- High variance queries (inconsistent performance = possible plan instability)
SELECT
query,
calls,
ROUND(mean_exec_time::numeric, 2) AS avg_ms,
ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
ROUND(stddev_exec_time / NULLIF(mean_exec_time, 0) * 100, 1) AS cv_pct
FROM pg_stat_statements
WHERE calls > 100
AND mean_exec_time > 1
ORDER BY (stddev_exec_time / NULLIF(mean_exec_time, 0)) DESC
LIMIT 10;

-- Reset statistics
SELECT pg_stat_statements_reset();

```

```
-- Or reset for a specific query:
SELECT pg_stat_statements_reset(userid, dbid, queryid)
FROM pg_stat_statements
WHERE query LIKE '%specific_pattern%';
```

PostGIS

PostGIS adds spatial/geographic capabilities to PostgreSQL.

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS postgis_topology;

-- PostGIS version
SELECT PostGIS_Version();

-- Geometry types
CREATE TABLE locations (
    id        SERIAL PRIMARY KEY,
    name      TEXT,
    position  GEOMETRY(POINT, 4326), -- 4326 = WGS 84 (GPS coordinates)
    region    GEOMETRY(POLYGON, 4326)
);

-- Insert with ST_MakePoint
INSERT INTO locations (name, position) VALUES
('Eiffel Tower', ST_SetSRID(ST_MakePoint(2.2945, 48.8584), 4326)),
('Big Ben',      ST_SetSRID(ST_MakePoint(-0.1246, 51.5007), 4326)),
('Colosseum',    ST_SetSRID(ST_MakePoint(12.4924, 41.8902), 4326)),
('Statue of Liberty', ST_SetSRID(ST_MakePoint(-74.0445, 40.6892), 4326));

-- Distance between two points (in meters)
SELECT
    a.name,
    b.name,
    ROUND(ST_Distance(
        a.position::geography,
        b.position::geography
    )::numeric / 1000, 2) AS distance_km
FROM locations a
JOIN locations b ON a.id < b.id
ORDER BY distance_km;

-- Find locations within 500km of Paris
SELECT name
FROM locations
WHERE ST_DWithin(
    position::geography,
    ST_SetSRID(ST_MakePoint(2.3522, 48.8566), 4326)::geography,
    500000 -- meters
);
```

```
-- Spatial index (essential for any PostGIS workload)
CREATE INDEX idx_locations_pos ON locations USING GIST (position);

-- Nearest neighbor search (K-NN)
SELECT name,
       ROUND(ST_Distance(
           position::geography,
           ST_SetSRID(ST_MakePoint(2.3522, 48.8566), 4326)::geography
       )::numeric / 1000, 2) AS km_from_paris
FROM locations
ORDER BY position <-> ST_SetSRID(ST_MakePoint(2.3522, 48.8566), 4326)
LIMIT 3;
```

pg_partman

Covered in detail in `03_partitioning.md` . Key commands:

```
CREATE EXTENSION IF NOT EXISTS pg_partman;

-- Auto-manage monthly partitions
SELECT partman.create_parent(
    p_parent_table => 'public.events',
    p_control       => 'created_at',
    p_interval      => 'monthly',
    p_premake       => 3
);

-- Maintenance
CALL partman.run_maintenance_proc();
```

TimescaleDB

TimescaleDB is a PostgreSQL extension for time-series data. It adds automatic time-based partitioning (hypertables) with compression and continuous aggregates.

```
CREATE EXTENSION IF NOT EXISTS timescaledb;

-- Create a hypertable (automatically partitioned by time)
CREATE TABLE metrics (
    time          TIMESTAMPTZ NOT NULL,
    device_id     INT NOT NULL,
    metric_name   TEXT NOT NULL,
    value         FLOAT
);

SELECT create_hypertable('metrics', 'time', chunk_time_interval => INTERVAL '1 day');
```

```

-- TimescaleDB-specific features:

-- Continuous Aggregate (like materialized view, auto-refreshes)
CREATE MATERIALIZED VIEW metrics_hourly
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 hour', time) AS bucket,
    device_id,
    AVG(value) AS avg_value,
    MAX(value) AS max_value,
    MIN(value) AS min_value
FROM metrics
GROUP BY 1, 2;

-- Data compression (reduces storage 90%+)
ALTER TABLE metrics SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'device_id'
);

SELECT compress_chunk(c.schema_name || '.' || c.table_name)
FROM show_chunks('metrics', older_than => INTERVAL '7 days') AS c;

-- Chunk info
SELECT * FROM timescaledb_information.chunks WHERE hypertable_name = 'metrics';

-- Time-bucket queries
SELECT
    time_bucket('1 hour', time) AS hour,
    device_id,
    AVG(value) AS avg_val
FROM metrics
WHERE time > now() - interval '24 hours'
GROUP BY 1, 2
ORDER BY 1;

```

pgvector

pgvector adds vector similarity search for machine learning embeddings (semantic search, recommendations, anomaly detection).

```

CREATE EXTENSION IF NOT EXISTS vector;

-- Create table with vector column
CREATE TABLE documents (
    id          SERIAL PRIMARY KEY,
    content     TEXT,
    embedding   vector(1536) -- 1536 dimensions for text-embedding-ada-002
);

```

```

-- Insert embeddings (normally from ML model)
INSERT INTO documents (content, embedding) VALUES
('PostgreSQL is a relational database', '[0.1, 0.2, 0.3, ...]::vector'),
('Machine learning uses neural networks', '[0.9, 0.1, 0.4, ...]::vector'),
('SQL is used for database queries', '[0.2, 0.3, 0.1, ...]::vector');

-- Cosine similarity search (semantic search)
SELECT id, content,
       1 - (embedding <=> '[0.15, 0.25, 0.35, ...]::vector') AS similarity
FROM documents
ORDER BY embedding <=> '[0.15, 0.25, 0.35, ...]::vector'
LIMIT 5;

-- <=> = cosine distance
-- <-> = L2 (Euclidean) distance
-- <#> = negative inner product

-- Create HNSW index (PostgreSQL 16+ / pgvector 0.5+)
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Or IVFFlat index (older, requires training)
CREATE INDEX ON documents USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- Set probes before search:
SET ivfflat.probes = 10;

-- Metadata filtering with vector search
SELECT content, embedding <-> query.vec AS distance
FROM documents,
     (SELECT '[0.1, 0.2, 0.3, ...]::vector' AS vec) query
WHERE category = 'database' -- relational filter applied first
ORDER BY distance
LIMIT 10;

-- Check index type
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'documents';

```

pg_trgm

`pg_trgm` provides trigram-based similarity matching — useful for fuzzy search, typo tolerance, and LIKE/ILIKE acceleration.

```

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Trigram similarity (0.0 to 1.0)
SELECT similarity('postgresql', 'postgres'); -- 0.769...
SELECT similarity('hello', 'world'); -- 0.0
SELECT similarity('catalog', 'catalogue'); -- 0.545...

-- Find similar strings

```

```

SELECT word, similarity(word, 'postgres')
FROM (VALUES ('postgresql'),('postgis'),('postgre'),('maria'),('mysql')) AS t(word)
WHERE similarity(word, 'postgres') > 0.3
ORDER BY similarity(word, 'postgres') DESC;

-- GIN trigram index for fast LIKE/ILIKE
CREATE TABLE users (
    id          SERIAL PRIMARY KEY,
    username TEXT NOT NULL,
    email       TEXT NOT NULL
);

CREATE INDEX idx_users_username_trgm ON users USING GIN (username gin_trgm_ops);
CREATE INDEX idx_users_email_trgm    ON users USING GIN (email gin_trgm_ops);

-- Now these use the index:
SELECT * FROM users WHERE username ILIKE '%john%';
SELECT * FROM users WHERE username % 'johnn'; -- % is similarity threshold operator

-- Set similarity threshold
SET pg_trgm.similarity_threshold = 0.3; -- default 0.3
SELECT * FROM users WHERE username % 'Johnathan';

-- Full-text search with trigrams (catches typos FTS doesn't)
SELECT username FROM users
WHERE username % 'postgresq'
ORDER BY similarity(username, 'postgresq') DESC;

-- pg_trgm vs FTS:
-- pg_trgm: handles typos, LIKE patterns, partial matches, no language config needed
-- FTS: handles stemming, stop words, ranking, phrase queries, better for documents

```

uuid-ossdp / gen_random_uuid

```

-- gen_random_uuid() is built-in since PostgreSQL 13 (no extension needed)
SELECT gen_random_uuid(); -- e.g., 550e8400-e29b-41d4-a716-446655440000

-- Use as default primary key
CREATE TABLE sessions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id     INT,
    data        JSONB,
    expires_at  TIMESTAMPTZ
);

INSERT INTO sessions (user_id, data) VALUES (1, '{"role": "admin"}');
SELECT * FROM sessions; -- id is auto-generated UUID

-- uuid-ossdp extension (for additional UUID versions)
CREATE EXTENSION IF NOT EXISTS "uuid-ossdp";

```



```

SELECT uuid_generate_v1(); -- time-based UUID (sequential, includes MAC)
SELECT uuid_generate_v4(); -- random UUID (same as gen_random_uuid)
SELECT uuid_generate_v5(uuid_ns_dns(), 'www.example.com'); -- name-based

-- UUID vs BIGSERIAL comparison:
-- UUID: globally unique without coordination, no sequential scan order, larger (16 bytes)
-- BIGSERIAL: auto-increment, sequential (good cache behavior), smaller (8 bytes)
-- For distributed systems: UUID. For single-node: BIGSERIAL usually better.

-- UUIDv7 (time-ordered, PostgreSQL 17+ or extension)
-- Solves index fragmentation problem of UUIDv4

```

hstore

`hstore` is a key-value store type. Largely superseded by `jsonb` but still used in legacy systems.

```

CREATE EXTENSION IF NOT EXISTS hstore;

CREATE TABLE product_attributes (
    id          SERIAL PRIMARY KEY,
    product_id  INT,
    attrs       HSTORE
);

INSERT INTO product_attributes (product_id, attrs) VALUES
(1, 'color => red, size => large, brand => Nike'),
(2, 'color => blue, material => cotton, weight => 200g');

-- Key access
SELECT attrs -> 'color' FROM product_attributes;
SELECT attrs -> ARRAY['color', 'size'] FROM product_attributes WHERE product_id = 1;

-- Key existence
SELECT * FROM product_attributes WHERE attrs ? 'brand';
SELECT * FROM product_attributes WHERE attrs ?& ARRAY['color', 'size'];

-- Containment
SELECT * FROM product_attributes WHERE attrs @> 'color => red';

-- Update
UPDATE product_attributes
SET attrs = attrs || 'discount => 10% '::hstore
WHERE product_id = 1;

-- Delete key
UPDATE product_attributes
SET attrs = delete(attrs, 'discount')
WHERE product_id = 1;

```

```
-- Convert to JSON
SELECT hstore_to_json(attrs) FROM product_attributes;

-- hstore vs jsonb:
-- hstore: flat key-value only, no nesting, lighter weight
-- jsonb: full JSON document, nested structures, better operator support
-- Recommendation: Use jsonb for new projects. hstore only in legacy systems.

-- GIN index for hstore
CREATE INDEX ON product_attributes USING GIN (attrs);
```

pg_cron

Schedules SQL jobs directly within PostgreSQL.

```
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Schedule examples (requires pg_cron in shared_preload_libraries)
SELECT cron.schedule('nightly-vacuum',      '0 3 * * *', 'VACUUM ANALYZE');
SELECT cron.schedule('refresh-matview',     '* /5 * * * *', 'REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales_summary');
SELECT cron.schedule('cleanup-old-sessions','0 0 * * *', 'DELETE FROM sessions WHERE expires_at < now()');

-- List scheduled jobs
SELECT * FROM cron.job;

-- Recent run history
SELECT
    jobid, jobname, status, start_time, end_time,
    end_time - start_time AS duration,
    return_message
FROM cron.job_run_details
ORDER BY start_time DESC
LIMIT 20;

-- Unschedule
SELECT cron.unschedule('nightly-vacuum');
SELECT cron.unschedule(jobid) FROM cron.job WHERE jobname = 'refresh-matview';
```

btree_gin / btree_gist

These extensions allow GIN and GiST indexes on data types that normally only support B-tree.

```
CREATE EXTENSION IF NOT EXISTS btree_gin;
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- Useful for composite indexes mixing regular columns and GIN/GiST types
```

```

-- Example: exclude constraint with btree_gist
CREATE TABLE reservations (
    id          SERIAL PRIMARY KEY,
    room_id     INT NOT NULL,
    during      TSTZRANGE NOT NULL,
    EXCLUDE USING GIST (room_id WITH =, during WITH &&)
    -- Prevents overlapping reservations for the same room
    -- Requires btree_gist for the integer room_id part
);

-- Test: insert non-overlapping
INSERT INTO reservations (room_id, during) VALUES
(1, '[2024-01-01 10:00, 2024-01-01 12:00)'),
(1, '[2024-01-01 12:00, 2024-01-01 14:00)'); -- OK: adjacent, not overlapping

-- Test: insert overlapping (will fail)
INSERT INTO reservations (room_id, during) VALUES
(1, '[2024-01-01 11:00, 2024-01-01 13:00)');
-- ERROR: conflicting key value violates exclusion constraint

```

Managing Extensions

```

-- List all extensions with versions
SELECT
    e.extname,
    e.extversion AS installed_version,
    ae.default_version AS latest_available,
    CASE WHEN e.extversion = ae.default_version THEN 'up-to-date'
          ELSE 'upgrade available' END AS status
FROM pg_extension e
JOIN pg_available_extensions ae ON ae.name = e.extname
ORDER BY e.extname;

-- Upgrade an extension
ALTER EXTENSION pg_stat_statements UPDATE;
ALTER EXTENSION pg_stat_statements UPDATE TO '1.10';

-- Install to specific schema
CREATE EXTENSION pg_trgm SCHEMA pg_catalog;

-- Extension ownership
ALTER EXTENSION pg_trgm OWNER TO admin_user;

-- Find all objects belonging to an extension
SELECT classid::regclass, objid, deptype
FROM pg_depend
WHERE refobjid = (SELECT oid FROM pg_extension WHERE extname = 'pg_trgm');

```

Production Monitoring Queries

```
-- Extension usage summary
SELECT
    e.extname,
    e.extversion,
    COUNT(d.objid) AS object_count
FROM pg_extension e
LEFT JOIN pg_depend d ON d.refobjid = e.oid AND d.deptype = 'e'
GROUP BY e.extname, e.extversion
ORDER BY object_count DESC;

-- pg_stat_statements: find queries that changed performance recently
-- (compare stats from different time windows)
-- Best done by pg_stat_statements_reset() + wait + compare

-- pg_trgm index effectiveness
SELECT
    indexname,
    tablename,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE indexdef LIKE '%gin_trgm_ops%'
ORDER BY idx_scan DESC;

-- pgvector index stats
SELECT schemaname, tablename, indexname, idx_scan
FROM pg_stat_user_indexes
WHERE indexdef LIKE '%hnsw%' OR indexdef LIKE '%ivfflat%';

-- PostGIS spatial queries in pg_stat_statements
SELECT query, calls, ROUND(mean_exec_time::numeric, 2) AS avg_ms
FROM pg_stat_statements
WHERE query ILIKE '%ST_%' OR query ILIKE '%GIST%'
ORDER BY mean_exec_time DESC LIMIT 10;
```

Common Mistakes

1. **Loading `pg_stat_statements` without `shared_preload_libraries`** — The extension must be in `shared_preload_libraries` (`postgresql.conf`) AND installed with `CREATE EXTENSION`. Missing either step results in no tracking.
2. **Using UUIDv4 as primary key with B-tree index on large tables** — Random UUIDs fragment the B-tree index and cause poor write performance. Consider UUIDv7 or BIGSERIAL for sequential access patterns.
3. **Not setting `ivfflat.probes` before pgvector search** — Default `probes=1` gives fast but inaccurate results. Set `SET ivfflat.probes = 10` for better recall. HNSW doesn't need this tuning.

4. **Using `hstore` for new projects** — It's flat key-value only and largely obsolete. Use `jsonb` for all new development.
 5. **Installing PostGIS on OLTP database** — PostGIS adds significant overhead. Use a separate database or schema for spatial data on high-transaction systems.
 6. **Forgetting to create spatial indexes for PostGIS** — `ST_Within`, `ST_DWithin`, and similar functions perform sequential scans without a GiST index.
-

Best Practices

1. **Always install `pg_stat_statements`** in every production database — it's the #1 diagnostic tool.
 2. **Use HNSW over IVFFlat** for pgvector — HNSW requires no training, is more accurate, and scales better.
 3. **Keep extensions up to date** — many include security fixes; `ALTER EXTENSION ... UPDATE` regularly.
 4. **Document which extensions each database uses** in your runbook — extensions are database-scoped.
 5. **Test extension upgrades in staging** before production — extension upgrades can change behavior.
-

Interview Questions & Answers

Q1: What is `pg_stat_statements` and why is it essential in production?

A: `pg_stat_statements` is a PostgreSQL extension that tracks execution statistics (calls, total time, mean time, rows, I/O blocks) for each unique query pattern. It's essential because it answers the fundamental question: "which queries are consuming the most database resources?" Without it, performance tuning is guesswork. It should be installed in every production PostgreSQL database.

Q2: What is the difference between HNSW and IVFFlat indexes in pgvector?

A: IVFFlat (Inverted File with Flat compression) divides the vector space into clusters (lists) during an offline training phase and searches nearby clusters. It requires running a build phase on existing data, the quality depends on `lists` and `probes` settings, and can't be built on empty tables. HNSW (Hierarchical Navigable Small World) builds a multi-layer graph structure, requires no training, supports incremental inserts, gives better recall at the same speed, but uses more memory and takes longer to build. HNSW is preferred for most production use cases.

Q3: When would you use `pg_trgm` vs full-text search?

A: Use `pg_trgm` when: you need fuzzy matching (typo tolerance), accelerating `LIKE '%pattern%'` queries, or comparing short strings for similarity. Use FTS when: searching document content, needing stemming/language support, phrase queries, ranking by relevance, or searching across multiple columns. They are complementary: `pg_trgm` for "sounds like" and `pg_trgm` for LIKE patterns; FTS for document search.

Q4: What is the spatial reference system (SRID) and why does it matter in PostGIS?

A: An SRID is a unique identifier for a spatial reference system (coordinate system). SRID 4326 is WGS 84 (GPS latitude/longitude). Using the wrong SRID means calculations are done in the wrong units. For distance calculations, always cast to `geography` type or ensure you're using a metric projection. `ST_Distance` on geometry returns degrees unless you use the geography type, which returns meters.

Q5: How does `pg_cron` work, and what are its limitations?

A: `pg_cron` runs as a background worker in PostgreSQL and executes scheduled SQL commands using cron syntax. It runs jobs as the database superuser by default. Limitations: (1) can only run SQL/functions (not OS commands), (2) requires `shared_preload_libraries` change and restart, (3) jobs run in the `postgres` database by default (configurable), (4) no dependency management between jobs, (5) if a job takes longer than its interval, the next run is skipped.

Q6: Why is UUIDv7 better than UUIDv4 as a primary key for large tables?

A: UUIDv4 is completely random, which means B-tree index inserts go to random positions, causing frequent page splits and index fragmentation. This hurts both write performance (many page splits) and read performance (poor cache locality). UUIDv7 encodes a timestamp prefix, making new UUIDs monotonically increasing (like BIGSERIAL) while still being globally unique without coordination. This preserves the global uniqueness advantage of UUIDs while avoiding B-tree fragmentation.

Q7: What `pg_stat_statements` columns help identify N+1 query problems?

A: Look at `calls` — an N+1 query has a very high call count for a simple query (e.g., 10,000 calls for a single-row lookup). Compare: a complex query with 100 calls might be less problematic than a trivial query with 100,000 calls. Also look at `rows / calls` ratio — an N+1 usually fetches 1 row per call. The combination of high calls + low `avg_rows` + low `avg_ms` is the N+1 signature.

Q8: How do you safely upgrade a PostgreSQL extension in production?

A: (1) Check `pg_available_extension_versions` for available versions. (2) Review the extension's changelog for breaking changes. (3) Test upgrade on a staging database first. (4) During a maintenance window or low-traffic period: `ALTER EXTENSION extension_name UPDATE TO 'new_version'`. (5) Some extensions require a session restart (`pg_stat_statements`) or even a server restart. (6) Verify functionality after upgrade with your test suite.

Exercises and Solutions

Exercise 1

Install `pg_stat_statements`, run 5 different query types, then write a SQL query that shows the top 3 queries by total execution time with their cache hit ratio.

Solution:

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Run various queries first, then:
SELECT
    LEFT(query, 80) AS query_preview,
    calls,
    ROUND(total_exec_time::numeric / 1000, 3) AS total_sec,
    ROUND(mean_exec_time::numeric, 2) AS avg_ms,
    ROUND(
        shared_blks_hit::numeric /
        NULLIF(shared_blks_hit + shared_blks_read, 0) * 100,
        1
    ) AS cache_hit_pct
```

```
FROM pg_stat_statements
WHERE query NOT LIKE '%pg_stat%'
      AND calls > 0
ORDER BY total_exec_time DESC
LIMIT 3;
```

Exercise 2

Create a `products` table with a trigram GIN index on the `description` column. Insert 100 rows with fake product names. Write a fuzzy search query that finds products similar to a misspelled query.

Solution:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE TABLE products_trgm (
    id          SERIAL PRIMARY KEY,
    name        TEXT,
    description TEXT
);

INSERT INTO products_trgm (name, description)
SELECT
    'Product ' || g,
    CASE (g % 5)
        WHEN 0 THEN 'High performance laptop computer for professionals'
        WHEN 1 THEN 'Wireless noise cancelling headphones premium audio'
        WHEN 2 THEN 'Ergonomic mechanical keyboard for gaming'
        WHEN 3 THEN 'Ultra HD monitor 4K display high resolution'
        ELSE 'Fast solid state drive SSD storage device'
    END
FROM generate_series(1, 100) g;

CREATE INDEX ON products_trgm USING GIN (description gin_trgm_ops);

-- Fuzzy search (handles typos like 'professionels' -> 'professionals')
SELECT name, description, similarity(description, 'professionels laptop') AS sim
FROM products_trgm
WHERE description % 'professionels laptop'
ORDER BY sim DESC
LIMIT 5;
```

Cross-References

- [02_full_text_search.md](#) — FTS vs pg_trgm comparison
- [01_jsonb_mastery.md](#) — jsonb vs hstore
- [03_partitioning.md](#) — pg_partman usage
- [07_indexes](#) — GIN/GiST index architecture (used by pg_trgm, pgvector, hstore)
- [02_pg_stat_views.md](#) (12_Production) — pg_stat_statements in monitoring context
- [04_observability_stack.md](#) (12_Production) — Prometheus with pgvector metrics